МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(национальный исследовательский университет)»

Институт №3 «Системы управления, информатика и электроэнергетика»

Кафедра 311 «Прикладные программные средства и математические методы»

Отчет

по дисциплине «Программная инженерия»

по теме «Электронная очередь МФЦ»

Выполнили:

Студенты гр. М3О-317Б-23

Макарова К. О

Твелова В. Б

Несмина К. Л

Москва

2026

Содержание

Содержание **2**
1. Спецификация требований к программе **3**
1.1 Назначение системы 3
1.2 Функциональные требования 3
1.3 Нефункциональные требования 3
1.4 Диаграмма прецедентов 4
2. Этап проектирования **4**
2.2 Диаграмма концептуальных классов 11
2.3 Диаграммы последовательностей 13
2.4 Диаграммы компонентов 17
3. Этап реализации **18**
3.1 Выбор языка программирования 18
3.2 Архитектура приложения 18
3.3 Листинг программы 21
4. Этап тестирования **35**
4.1 План тестирования 35
4.2 Тестовые сценарии 35
4.3 Протокол тестирования 35
4.4 Скриншоты работы программы 35
5. ЗАКЛЮЧЕНИЕ **40**

1. Спецификация требований к программе

1.1 Назначение системы

Система электронной очереди предназначена для:

- выдачи талонов клиентам;
- управления очередью;
- вызова клиентов операторами;
- отображения информации на цифровом табло;
- предварительной записи;
- обработки оплаты госпошлины.

1.2 Функциональные требования

Система должна обеспечивать следующие функции:

	Требование
R-01	Получение талона через терминал
R-02	Предварительная запись на приём
R-03	Отслеживание состояния очереди
R-04	Вызов клиента оператором
R-05	Оплата госпошлины
R-06	Завершение обслуживания
R-07	Перенаправление клиента
R-08	Фиксация неявки

Функциональные требования подробно описаны в сценариях использования (Use Cases) файла проекта.

1.3 Нефункциональные требования

Система должна:

1. обеспечивать непрерывную работу терминалов;
2. поддерживать хранение данных в БД;

3. иметь интуитивно понятный интерфейс;
4. обеспечивать защиту данных пользователей.

1.4 Диаграмма прецедентов

Акторы:

1. Клиент
2. Оператор

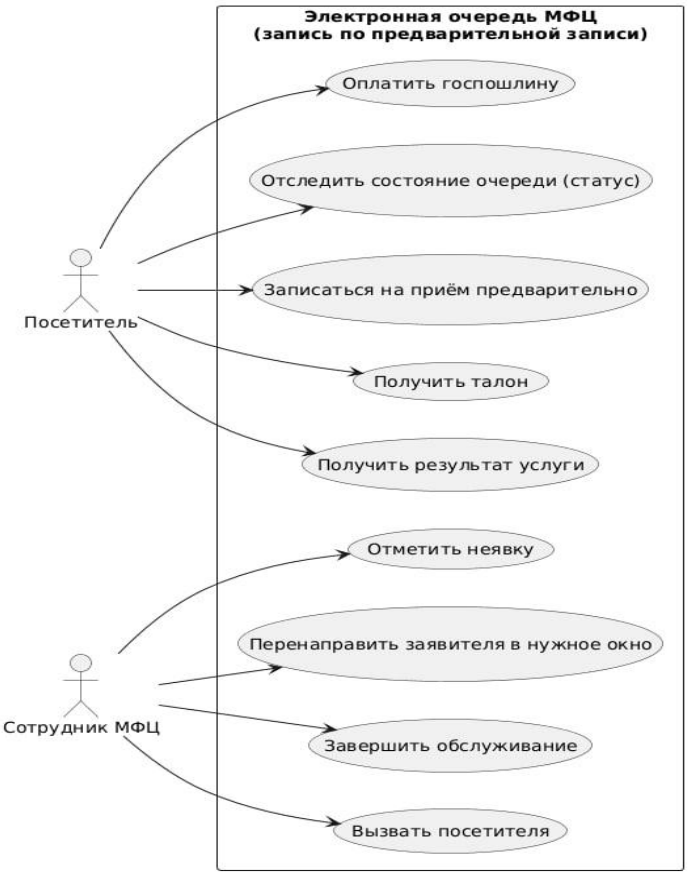
Основные прецеденты:

Клиент:

1. Получить талон
2. Записаться на приём
3. Отследить состояние очереди
4. Оплатить госпошлину
5. Получить результат услуги

Сотрудник:

1. Вызвать клиента
2. Отметить неявку
3. Перенаправить клиента
4. Завершить обслуживание



2. Этап проектирования

2.1 Таблица прецедентов

UC-01: Получение талона

Поле	Описание
Идентификатор	UC-01
Название	Получение талона
Акторы	Клиент
Предусловие	Терминал включен и отображает стартовый экран. В базе данных заведены доступные типы услуг.
Триггер	Клиент начинает взаимодействие с экраном терминала.
Основной сценарий	1. Терминал отображает перечень доступных типов услуг. 2. Клиент выбирает необходимую услугу 3. Терминал передает команду контроллеру талонов. 4. Контроллер создает новый талон, фиксирует время, ставит статус «В очереди» и добавляет его в очередь. 5. Терминал печатает бумажный талон с номером.
Альтернативные сценарии	2a. Выбранная услуга требует предварительной записи — терминал выводит уведомление и прерывает процесс создания талона. 4a. Очередь переполнена или выдача талонов завершена — контроллер прерывает операцию, терминал выводит извинение и предлагает прийти позже. 5a. В терминале закончилась бумага — терминал выводит номер талона крупно на экран, предлагая клиенту запомнить или сфотографировать его.
Постусловие	В памяти сохранен новый талон. Клиент физически (или на экране) получил свой номер талона и ожидает вызова.

UC-02: Записаться на прием предварительно

Поле	Описание
Идентификатор	UC-02
Название	Записаться на приём предварительно
Актёры	Клиент
Предусловие	Интерфейс предварительной записи (веб-портал или терминал) доступен. В базе данных настроены типы услуг и расписание.
Триггер	Клиент выбирает пункт «Записаться на приём предварительно» в интерфейсе.
Основной сценарий	1. Интерфейс запрашивает у контроллера и отображает перечень доступных типов услуг. 2. Клиент выбирает необходимую услугу. 3. Контроллер вычисляет свободные интервалы, а интерфейс отображает доступные дату и время в очереди. 4. Клиент выбирает удобную дату и время в очереди.

	5. Интерфейс запрашивает расширенные контактные данные (ФИО, номер телефона, email). 6. Клиент вводит свои данные. 7. Контроллер проверяет корректность введенных контактных данных. 8. Контроллер создает талон, присваивает ему статус «Предварительная запись» и фиксирует выбранную дату и время. 9. Интерфейс выводит подтверждение: номер созданного талона, дату и время.
Альтернативные сценарии	3а. Нет доступных дат для выбранной услуги — контроллер возвращает пустой список, интерфейс выводит сообщение об отсутствии свободного времени и предлагает выбрать другую услугу. 6а. Введены некорректные контактные данные (например, неверный формат номера телефона или email) — контроллер отклоняет валидацию, интерфейс подсвечивает ошибку и просит исправить данные. 8а. Выбранное время только что заняли (конкурентная запись другого клиента) — контроллер отклоняет операцию, интерфейс выводит сообщение об ошибке и просит выбрать другое время.
Постусловие	В базе данных сохранен талон предварительной записи, привязанный к конкретной дате, времени и контактным данным. Клиент получил подтверждение с номером талона.

UC-03: Отследить состояние очереди

Поле	Описание
Идентификатор	UC-03
Название	Отследить состояние очереди
Акторы	Клиент
Предусловие	У клиента имеется зарегистрированный номер талона. Информационные табло (общее цифровое табло в зале и/или экраны над окнами) включены и работают.
Триггер	Клиент обращает внимание на информационное табло для проверки своей очереди.
Основной сценарий	1. Контроллер непрерывно и автоматически передает актуальную информацию о статусе талонов на цифровое табло. 2. Табло отображает список текущих талонов и номера назначенных окон. 3. Клиент визуально сверяет номер своего талона с информацией на экране. 4. Клиент определяет свое положение: ожидает очереди (если номера еще нет) или приглашен к конкретному окну (если номер появился на экране).
Альтернативные сценарии	1а. Индивидуальная проверка через терминал (или сайт) — если клиент хочет узнать точное количество человек перед ним или пропустил свой номер, он может подойти к терминалу, ввести номер талона, и система выведет его подробный статус на экран
Постусловие	Клиент проинформирован о текущем статусе своего талона и номере назначенного окна. Состояние данных в системе не изменяется (операция только для чтения).

UC-04: Вызвать посетителя

Поле	Описание

Идентификатор	UC-04
Название	Вызвать клиента
Акторы	Оператор
Предусловие	Оператор авторизован в системе и привязан к конкретному рабочему окну. В очереди присутствует хотя бы один ожидающий талон.
Триггер	Оператор нажимает кнопку вызова следующего клиента на своей панели программы.
Основной сценарий	1. Оператор инициирует вызов в интерфейсе панели программы. 2. Интерфейс передает команду контроллеру очереди. 3. Контроллер обращается к списку ожидания и извлекает первый по времени или приоритету талон. 4. Контроллер меняет статус этого талона на «Вызван» и связывает его с текущим окном оператора. 5. Контроллер обновляет панель программы, чтобы оператор увидел номер текущего талона и данные записи на своем экране. 6. Одновременно контроллер отправляет команду на цифровое табло в зале. 7. Цифровое табло отображает номер талона и номер назначенного окна, сопровождая это звуковым оповещением.
Альтернативные сценарии	3а. В очереди нет ожидающих талонов — контроллер возвращает пустой ответ, панель программы отображает сообщение «Очередь пуста, ожидающих клиентов нет». 6а. Цифровое табло временно недоступно из-за сбоя сети — контроллер фиксирует ошибку, но операция вызова не прерывается. Панель программы уведомляет оператора о сбое табло, чтобы он мог вызвать клиента голосом.
Постусловие	Статус талона изменен на «Вызван», талон аппаратно привязан к рабочему окну. Клиент оповещен и должен подойти к окну для начала обслуживания.

UC-05: Оплатить госпошлину

Поле	Описание
Идентификатор	UC-05
Название	Оплатить госпошлину
Акторы	Клиент
Предусловие	Клиент выбрал услугу, которая предусматривает взимание пошлины. Терминал исправен и имеет связь с банковским платежным шлюзом.
Триггер	Клиент выбирает действие «Оплатить» на экране терминала (в процессе получения талона или как отдельную функцию).
Основной сценарий	1. Контроллер определяет необходимость оплаты и размер суммы для выбранной услуги. 2. Терминал отображает клиенту сумму к оплате и предлагает способы оплаты (банковская карта, QR-код и т.д.). 3. Клиент производит оплату, взаимодействуя со считывателем терминала. 4. Внешний банковский шлюз присылает контроллеру подтверждение об успешной транзакции.

	5. Контроллер фиксирует успешную оплату (связывает факт оплаты с формируемым талоном). 6. Терминал печатает квитанцию об оплате (или выдает бумажный талон с отметкой об уплаченной пошлине).
Альтернативные сценарии	1а. Для выбранной услуги госпошлина не предусмотрена — контроллер сообщает об этом терминалу, который автоматически пропускает этот шаг и переходит к выдаче талона. 4а. Оплата отклонена банком (недостаточно средств, ошибка чтения карты) — контроллер получает отказ, терминал выводит сообщение об ошибке транзакции и предлагает повторить попытку или выбрать другой способ оплаты. 4б. Превышено время ожидания ответа от клиента (тайм-аут) — терминал отменяет процесс создания талона и возвращается на стартовый экран.
Постусловие	Госпошлина успешно оплачена, факт оплаты зафиксирован контроллером в базе данных. Клиент получил подтверждающий документ.

UC-06: Получить результат услуги

Поле	Описание
Идентификатор	UC-06
Название	Получить результат услуги
Акторы	Клиент, Оператор
Предусловие	У клиента есть талон со статусом «Вызван». Клиент физически находится перед назначенным рабочим окном.
Триггер	Клиент передает свой номер талона или документ оператору для получения услуги.
Основной сценарий	1. Оператор сверяет номер талона и данные клиента с информацией, отображаемой в панели программы. 2. Оператор физически выдает подготовленные документы или оказывает консультацию. 3. По завершении обслуживания оператор нажимает кнопку завершения в своей панели программы. 4. Интерфейс передает команду контроллеру очереди. 5. Контроллер обновляет статус талона на «Завершен». 6. Контроллер переводит статус рабочего окна в значение «Свободно».
Альтернативные сценарии	1а. Клиент подошел не к своему окну или статус его талона отличный от «Вызван» — оператор устно уведомляет клиента об ошибке. Состояние системы не меняется. 2а. Не хватает документов или результат услуги еще не готов — оператор фиксирует отказ в панели программы. Контроллер меняет статус талона на «Отклонен/Не оказано», после чего переводит статус окна в «Свободно». **Вариант неявки:** Если клиент вообще не подошел к окну после вызова, оператор инициирует прецедент «Отметить неявку» (вынесен в отдельный сценарий).
Постусловие	Жизненный цикл талона завершен (установлен финальный статус). Рабочее окно освобождено и готово к приему следующего вызова.

UC-07: Завершить обслуживание

Поле	Описание
Идентификатор	UC-07
Название	Завершить обслуживание
Акторы	Оператор
Предусловие	Рабочее окно находится в статусе «Занято». К окну привязан талон со статусом «Обслуживается» (или «Вызван»).
Триггер	Оператор нажимает кнопку «Завершить» в интерфейсе своей панели программы.
Основной сценарий	1. Оператор выбирает итог обслуживания (например, «Успешно») и нажимает кнопку завершения в панели программы. 2. Интерфейс передает команду и выбранный итог контроллеру очереди. 3. Контроллер обновляет статус текущего талона на «Завершен». 4. Контроллер переводит статус рабочего окна в «Свободно». 5. Контроллер обновляет панель программы: данные закрытого талона очищаются с экрана, и разблокируется кнопка для вызова следующего клиента.
Альтернативные сценарии	1а. Обслуживание невозможно завершить положительно (например, клиент забыл паспорт) — оператор выбирает итог «Отказ». Контроллер меняет статус талона на «Отклонен» и освобождает окно. 1б. Требуется перенаправление — оператор выбирает в интерфейсе другую услугу. Контроллер меняет статус талона на «Перенаправлен» и возвращает его в список ожидания, после чего освобождает текущее окно (переход к прецеденту «Перенаправить клиента»). 3а. Ошибка связи с базой данных — контроллер не может сохранить финальный статус талона. Панель программы выводит предупреждение и предлагает нажать кнопку завершения еще раз.
Постусловие	Работа с текущим талоном закончена, его статус зафиксирован в базе данных. Рабочее окно освобождено, система готова к вызову следующего клиента.

UC-08: Перенаправить заявителя в нужное окно

Поле	Описание
Идентификатор	UC-08
Название	Перенаправить заявителя в нужное окно
Акторы	Оператор
Предусловие	Талон находится в статусе «Обслуживается», текущее рабочее окно имеет статус «Занято». Клиент находится у окна оператора.
Триггер	Оператор нажимает кнопку «Перенаправить» в интерфейсе панели программы.

Основной сценарий	1. Оператор выбирает функцию перенаправления в панели программы. 2. Интерфейс запрашивает у контроллера очереди список доступных услуг или конкретных окон и отображает их на экране. 3. Оператор выбирает нужную услугу (или номер нового окна) и нажимает подтверждение. 4. Контроллер обновляет талон: привязывает его к новой услуге, повышает его приоритет (чтобы клиент оказался в начале новой очереди) и возвращает статус в положение «В очереди». 5. Контроллер переносит этот талон обратно в список ожидания. 6. Контроллер переводит статус текущего окна в «Свободно», очищает данные талона с экрана панели программы и разблокирует кнопку для вызова следующего клиента. 7. Оператор устно сообщает клиенту, что талон переведен и нужно дождаться нового вызова на цифровом табло.
Альтернативные сценарии	2а. В МФЦ сейчас нет специалистов по выбранной услуге (нет активных окон для данной услуги) — контроллер передает предупреждение, панель программы выводит сообщение «Специалисты по данной услуге недоступны». Оператор информирует клиента устно. 4а. Ошибка сохранения данных контроллером — изменения не фиксируются, панель программы выдает сообщение об ошибке, оператор повторяет транзакцию.
Постусловие	Талон с измененными параметрами услуги и повышенным приоритетом возвращен в список ожидания. Текущее рабочее окно освобождено.

UC-09: Отметить неявку

Поле	Описание
Идентификатор	UC-09
Название	Отметить неявку
Акторы	Оператор
Предусловие	Талон находится в статусе «Вызван» и привязан к конкретному рабочему окну. Установленное время ожидания истекло.
Триггер	Оператор нажимает кнопку «Отметить неявку» в интерфейсе панели программы.
Основной сценарий	1. Оператор визуально убеждается, что клиент не явился к окну, и выбирает действие «Отметить неявку» в панели программы. 2. Интерфейс передает команду контроллеру очереди. 3. Контроллер фиксирует факт неявки, изменяя статус текущего талона на финальное значение «Неявка». 4. Контроллер переводит статус рабочего окна в «Свободно».

	5. Контроллер обновляет интерфейс панели программы: данные пропущенного талона скрываются, и оператору становится доступна кнопка для вызова следующего клиента.
Альтернативные сценарии	1а. Клиент подошел в последний момент — клиент явился до того, как оператор подтвердил неявку в программе. Оператор отменяет свое действие и переходит к обслуживанию (сценарий «Получить результат услуги»). 3а. Ошибка сохранения статуса — контроллер не может обновить статус талона в базе данных. Панель программы выводит сообщение об ошибке и предлагает оператору повторить действие.
Постусловие	Жизненный цикл талона завершен со статусом «Неявка». Рабочее окно освобождено, система готова к вызову следующего человека.

2.2 Диаграмма концептуальных классов

Акторы

Класс	Атрибуты
Клиент (Client)	ID, Категория (Обычный/Льготный), Имя, Фамилия, Отчество, Телефон
Оператор (Operator)	ID, ФИО, Роль

Концептуальные классы

Класс	Атрибуты
Услуга (ServiceType)	ID, Название, Среднее время, Приоритет
Талон (Ticket)	Номер, Время создания, Статус (В очереди, Вызван, Завершен)
ОкноОчереди (QueueWindow)	Номер окна, Статус (Свободно, Занято)
Очередь (Queue)	ID услуги, Список талонов, Текущая длина

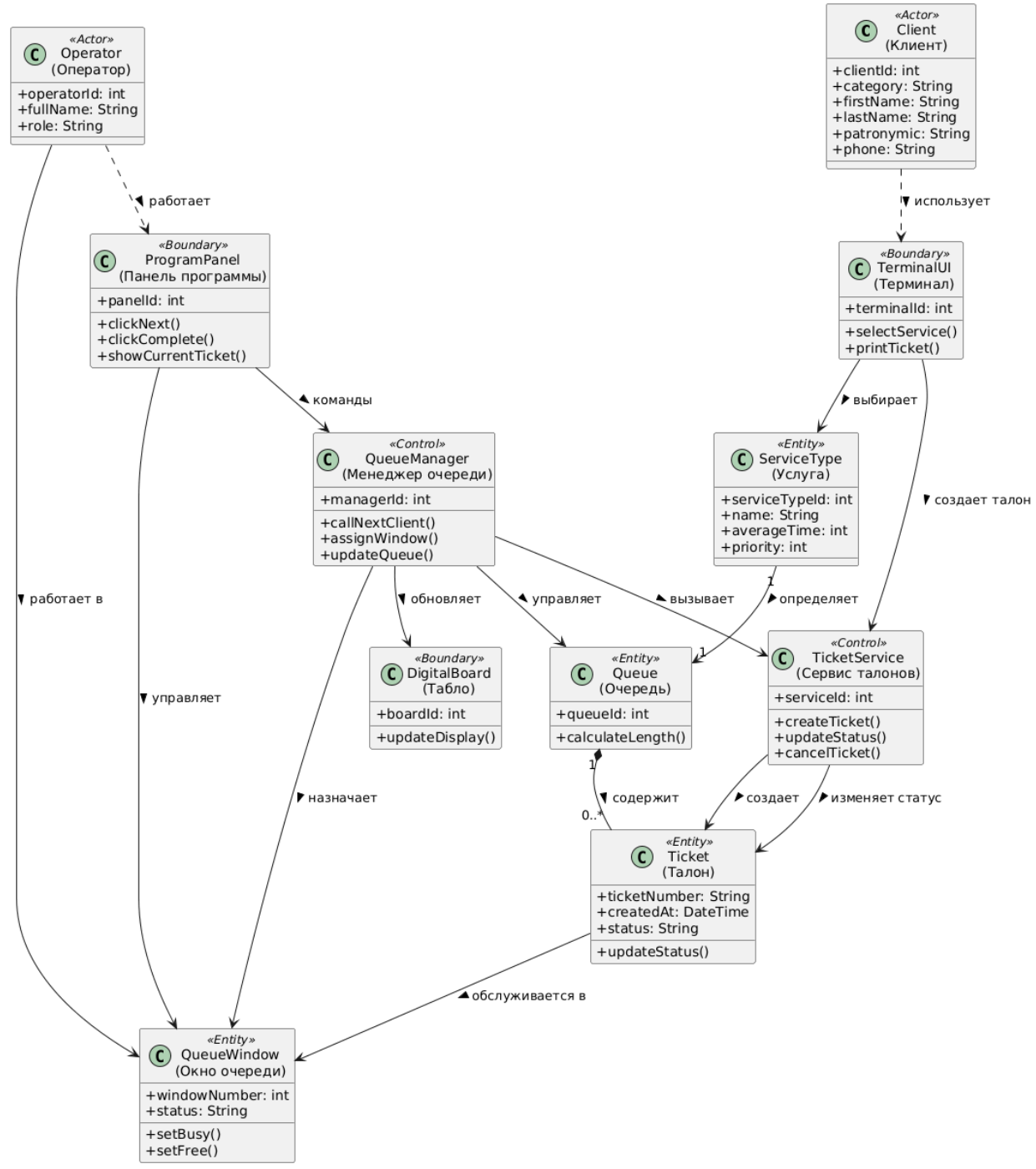
Интерфейсные классы

Класс	Атрибуты
Терминал (TerminalUI)	ID
Окно (DigitalBoard)	ID
Панель (ProgramPanel)	ID

Контроллер классы

Класс	Атрибуты
МенеджерОчереди(QueueManager)	ID
ТалонСервисTicketService	ID

Диаграмма классов



Связи между классами

Класс 1	Кратность	Класс 2	Тип связи	Пояснение

Client	0..*	TerminalUI	ассоциация	Клиенты используют терминалы для получения талона
Operator	1..*	ProgramPanel	ассоциация	Оператор работает через панель управления
TerminalUI	1	QueueManager	зависимость (dependency)	Терминал передаёт запросы менеджеру очереди
TerminalUI	1	TicketService	зависимость	Терминал инициирует создание талона
TerminalUI	0..*	ServiceType	ассоциация	Клиент выбирает услугу в терминале
TicketService	1	Ticket	создание	Сервис создаёт и обновляет талоны
Queue	1	0..* Ticket	композиция	Очередь содержит талоны
ServiceType	1	1 Queue	ассоциация	Каждая услуга имеет свою очередь
QueueManager	1	0..* QueueWindow	ассоциация	Менеджер управляет окнами обслуживания
QueueManager	1	Queue	ассоциация	Управление очередями
QueueManager	1	DigitalBoard	зависимость	Обновление табло вызова
Ticket	0..1	QueueWindow	ассоциация	Талон обслуживается в окне
ProgramPanel	1	1..* QueueWindow	ассоциация	Панель управляет окнами обслуживания
QueueWindow	1	ProgramPanel	ассоциация	Окно использует панель оператора

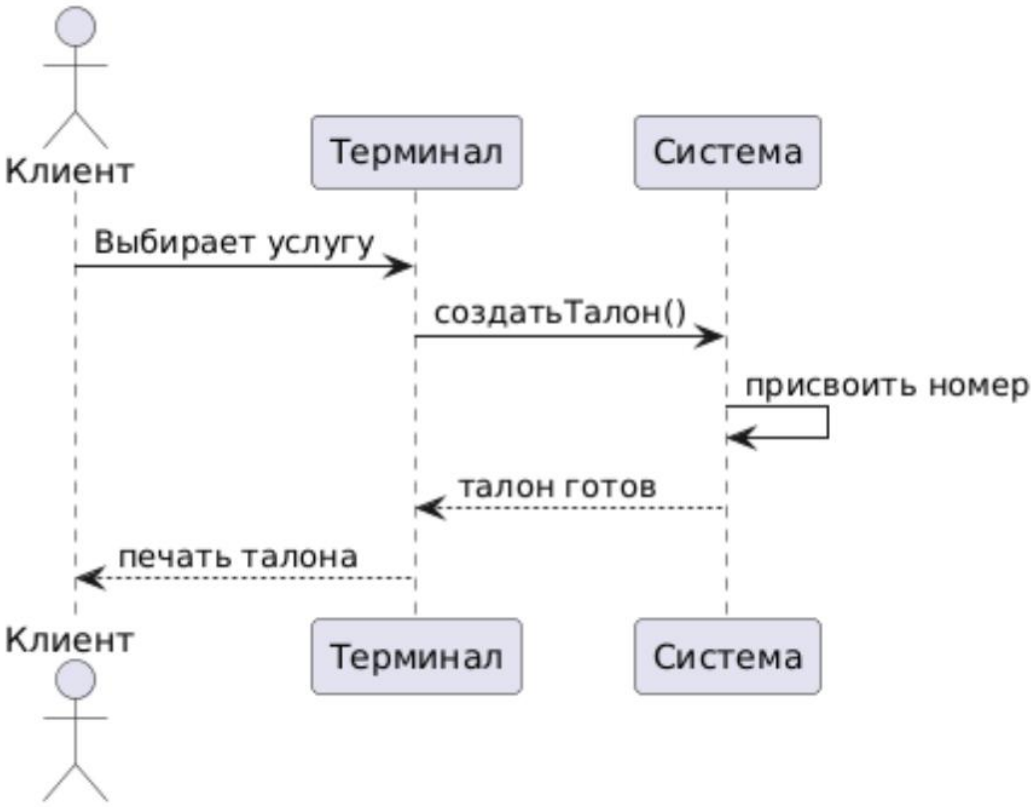
2.3 Диаграммы последовательностей

Каждая диаграмма описывает взаимодействие объектов для соответствующего Use Case.

SD-01: Получение талона (UC-01)

Участники: Терминал, КонтроллерТалонов, СервисОчереди, РепозиторийТалонов, Принтер

№	Сообщение
1	Терминал → КонтроллерТалонов : выбратьУслугу(serviceId)
2	КонтроллерТалонов → СервисОчереди : проверитьОчередь(serviceId)
3	СервисОчереди → КонтроллерТалонов : очередьДоступна
4	КонтроллерТалонов → РепозиторийТалонов : создатьТалон(serviceId, статус="В очереди")
5	РепозиторийТалонов → КонтроллерТалонов : талонСоздан(ticketNumber)
6	КонтроллерТалонов → Принтер : печатать(ticketNumber)
7	КонтроллерТалонов → Терминал : показатьТалон(ticketNumber)

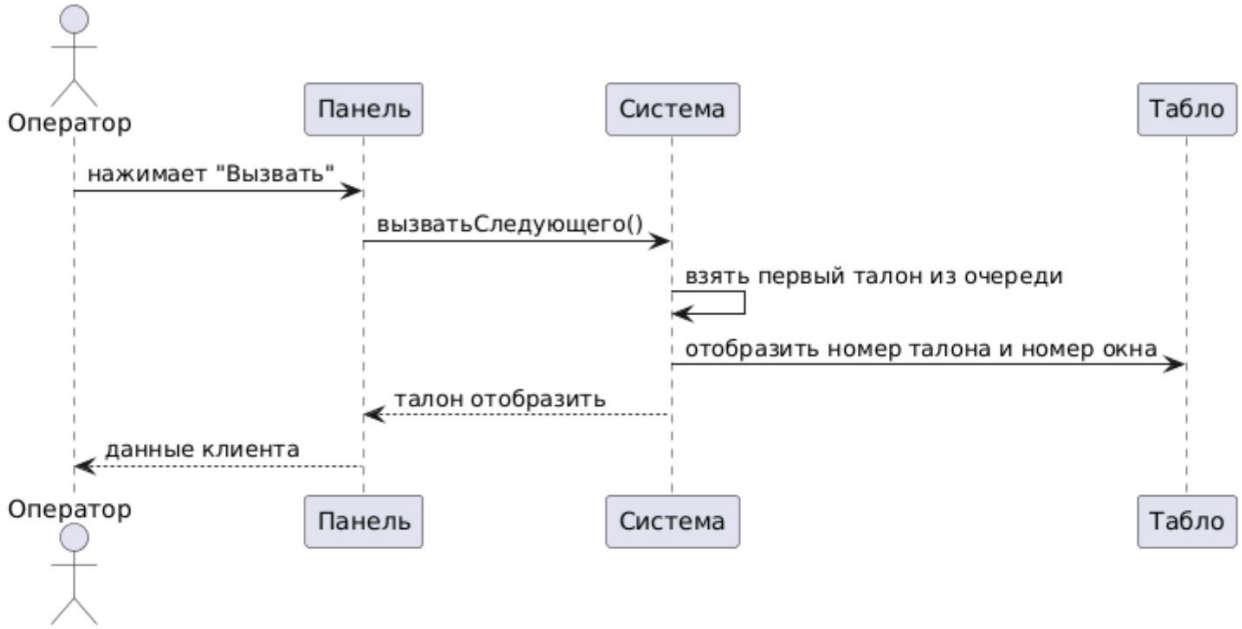


SD-02: Вызов клиента оператором (UC-04)

Участники: ПанельОператора, КонтроллерОчереди, Очередь, Талон, ОкноОчереди, ЦифровоеТабло

№	Сообщение
1	ПанельОператора → КонтроллерОчереди : вызватьСледующего(windowId)

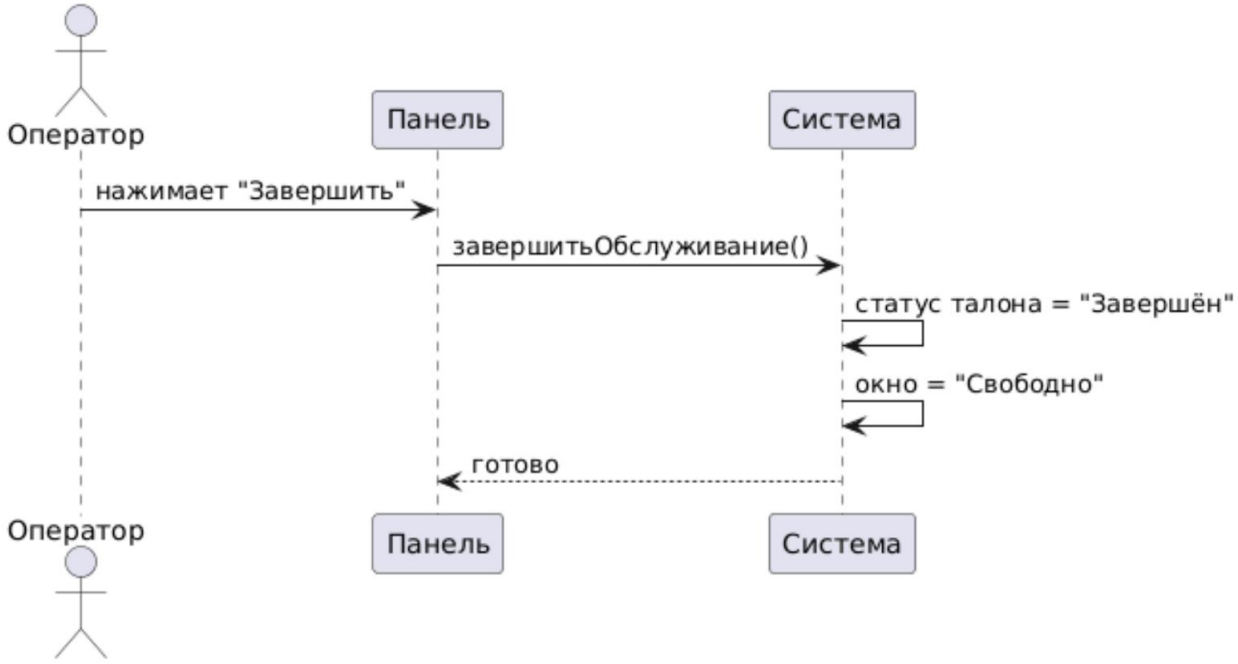
2	КонтроллерОчереди → Очередь : извлечьПервый(serviceId)
3	Очередь → КонтроллерОчереди : Талон(ticketId)
4	КонтроллерОчереди → Талон : установитьСтатус("Вызван")
5	КонтроллерОчереди → ОкноОчереди : привязатьТалон(windowId, ticketId)
6	КонтроллерОчереди → ПанельОператора : показатьТалон(номер)
7	КонтроллерОчереди → ЦифровоеТабло : обновить(номер, окно)



SD-03: Завершение обслуживания (UC-06 + UC-07)

Участники: ПанельОператора, КонтроллерОчереди, Талон, ОкноОчереди

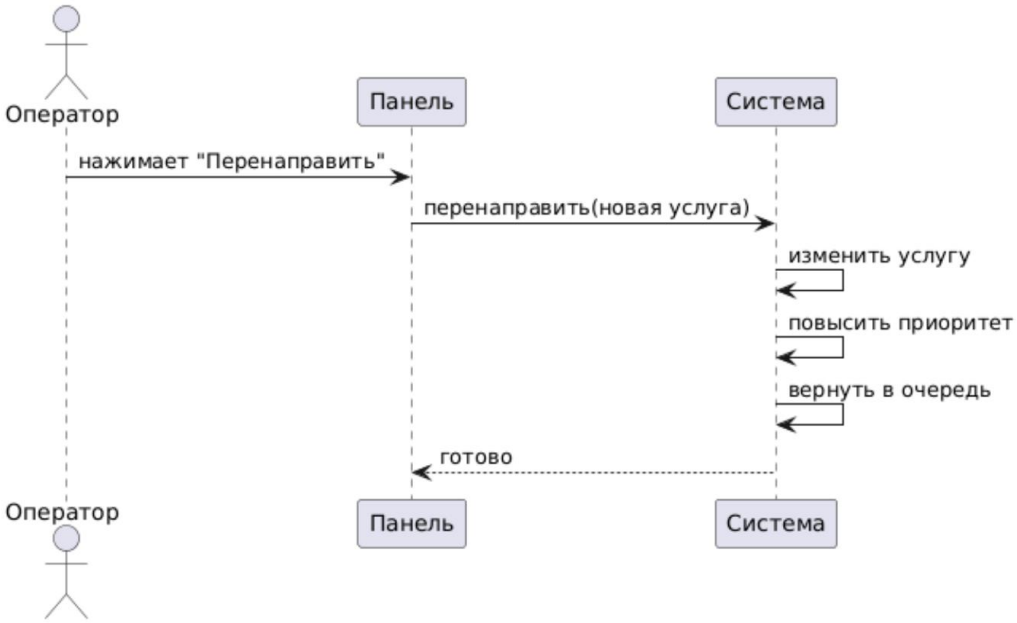
№	Сообщение
1	ПанельОператора → КонтроллерОчереди : завершитьОбслуживание(ticketId)
2	КонтроллерОчереди → Талон : установитьСтатус("Завершён")
3	КонтроллерОчереди → ОкноОчереди : освободить(windowId)
4	КонтроллерОчереди → ПанельОператора : окноСвободно()



SD-04: Перенаправление заявителя (UC-08)

Участники: ПанельОператора, КонтроллерОчереди, Талон, Очередь, ОкноОчереди

№	Сообщение
1	ПанельОператора → КонтроллерОчереди : перенаправить(ticketId, новыйServiceId)
2	КонтроллерОчереди → Талон : изменитьУслугу(новыйServiceId)
3	КонтроллерОчереди → Талон : повыситьПриоритет()
4	КонтроллерОчереди → Талон : установитьСтатус("В очереди")
5	КонтроллерОчереди → Очередь : добавитьВОчередь(ticketId)
6	КонтроллерОчереди → ОкноОчереди : освободить(windowId)
7	КонтроллерОчереди → ПанельОператора : подтверждение()

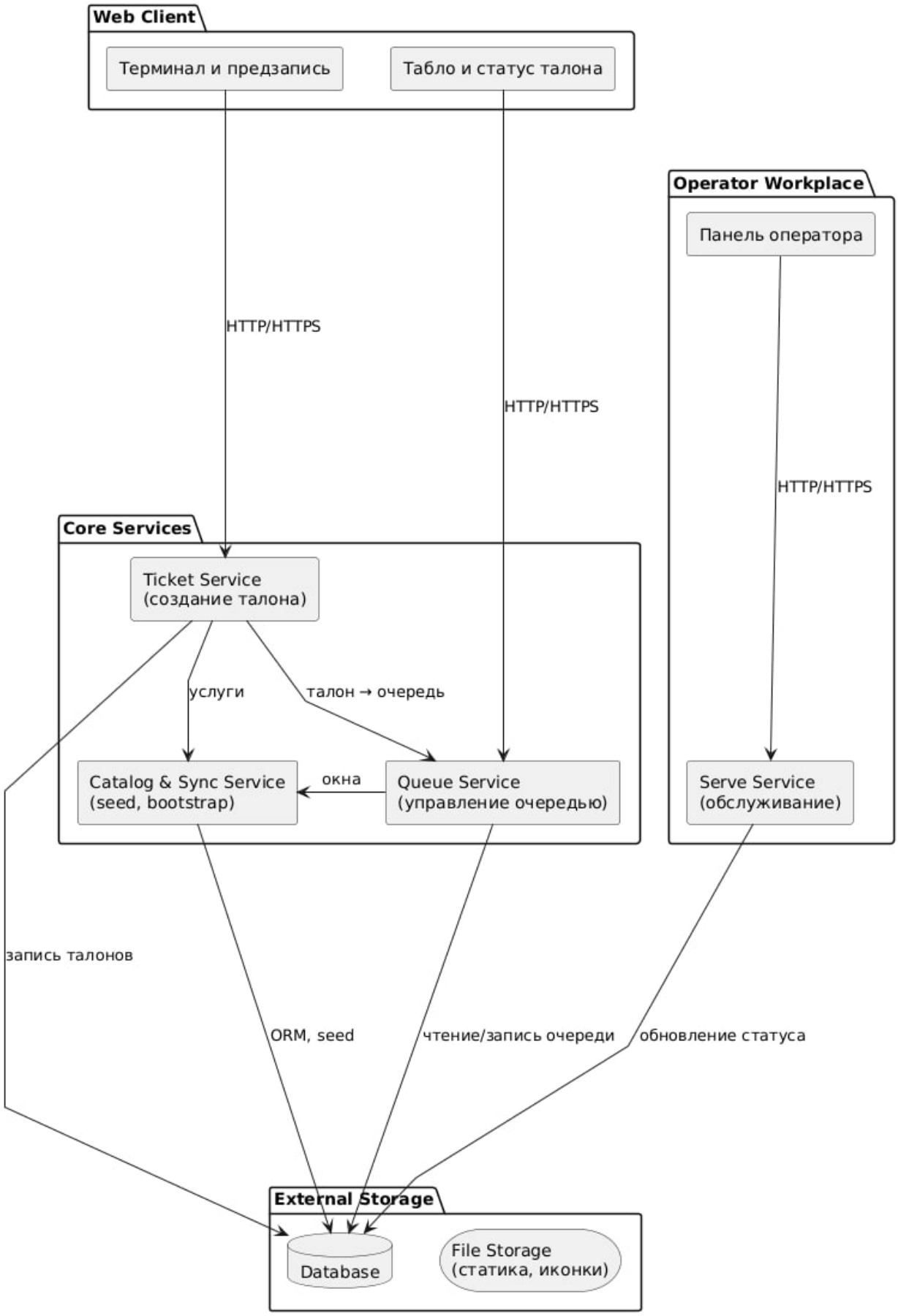


4. Сводная таблица акторов

Актор	Описание
Клиент (Client)	Физическое лицо, обращающееся в МФЦ за получением услуг. Является инициатором процесса обслуживания. Взаимодействует с системой через интерфейсные классы: использует терминал (TerminalUI) для выбора услуги и генерации талона, а также отслеживает свою очередь и номер назначенного окна с помощью цифрового табло (DigitalBoard).
Оператор (Operator)	Сотрудник МФЦ, осуществляющий приём заявителей в назначенном окне (QueueWindow). Взаимодействует с системой исключительно через свой рабочий интерфейс — панель программы (ProgramPanel). Через неё отправляет команды контроллеру (QueueManager) на вызов следующего клиента, завершение обслуживания, перенаправление талона или фиксацию неявки.

2.4 Диаграммы компонентов

5. Диаграмма компонентов (электронная очередь МФЦ)



3. Этап реализации

3.1 Выбор языка программирования

Для разработки системы электронной очереди был выбран язык программирования Python. Данный язык обладает простым и понятным синтаксисом, что позволяет быстро разрабатывать и поддерживать программное обеспечение.

В качестве серверного фреймворка использовался FastAPI, который обеспечивает высокую производительность приложения, удобную реализацию REST API и автоматическую генерацию документации. Использование FastAPI позволило реализовать взаимодействие между клиентской частью системы, терминалами и модулем управления очередью.

В качестве визуализации сайта и написания фронтенд части использовался язык HTML

Для работы с базой данных применялась библиотека SQLAlchemy, реализующая ORM-подход. Это позволило удобно описывать модели данных в виде классов Python и выполнять операции с базой данных без написания большого количества SQL-запросов.

В качестве системы управления базами данных была выбрана SQLite. Данная СУБД не требует отдельной установки сервера, проста в настройке и подходит для разработки и тестирования небольших информационных систем.

3.2 Архитектура приложения

Разработанная система построена по клиент-серверной архитектуре. Основная логика обработки запросов реализована на серверной стороне с использованием FastAPI.

Архитектура приложения включает следующие основные компоненты:

- пользовательский интерфейс для взаимодействия клиентов и операторов с системой;
- серверное API, реализующее бизнес-логику управления электронной очередью;
- модуль управления талонами и очередями;
- модуль обработки статусов обслуживания;
- базу данных SQLite для хранения информации о клиентах, талонах, услугах и очередях.

Для взаимодействия с базой данных использовалась библиотека SQLAlchemy, обеспечивающая связь между объектами программы и таблицами базы данных.

Система поддерживает:

- создание и хранение талонов в очереди;

- управление очередями;
- вызов клиентов операторами;
- изменение статусов обслуживания;
- предварительную запись на приём;

3.3 Иерархия файлов программы

```
MFC_electronic_queue-master/
│
├── mfc_queue/
│   │
│   ├── routers/
│   │   ├── __init__.py
│   │   ├── auth.py
│   │   ├── client.py
│   │   └── operator.py
│   │
│   ├── static/
│   │   ├── icons/
│   │   │   └── services/
│   │   │       ├── cadastre.svg
│   │   │       ├── default.svg
│   │   │       ├── egrn.svg
│   │   │       ├── encumbrance.svg
│   │   │       ├── family-cert.svg
│   │   │       ├── kindergarten.png
│   │   │       ├── license.svg
│   │   │       ├── passport.svg
│   │   │       ├── registration.svg
│   │   │       ├── social.svg
│   │   │       ├── utilities.svg
│   │   │       └── vehicle.svg
│   │   ├── family-group.png
│   │   └── transport.png
│   │
│   ├── templates/
│   │   ├── auth_login.html
│   │   ├── board.html
│   │   ├── client_dashboard.html
│   │   ├── layout.html
│   │   ├── operator_dashboard.html
│   │   ├── operator_panel.html
│   │   ├── pay.html
│   │   ├── prebook.html
│   │   ├── prebook_ok.html
│   │   ├── status.html
│   │   └── terminal.html
│   │
│   ├── __init__.py
│   ├── __main__.py
│   ├── bootstrap.py
│   ├── database.py
│   ├── domain.py
│   ├── main.py
│   ├── models.py
│   ├── seed.py
│   ├── services.py
│   ├── state.py
│   ├── store.py
│   └── templating.py
│
├── mfc_queue.db
└── requirements.txt
```

Назначение основных каталогов

Каталог / файл	Назначение
routers	Маршруты и обработчики HTTP-запросов
templates	HTML-шаблоны пользовательского интерфейса
static	Статические файлы: изображения, иконки
models.py	ORM-модели SQLAlchemy
services.py	Бизнес-логика приложения
database.py	Настройка подключения к SQLite
main.py	Точка запуска приложения
requirements.txt	Список используемых библиотек

3.3 Листинг программы

1. Запуск приложения (main.py)

```
router = APIRouter(tags=["auth"])
#Главная страница
@router.get("/", response_class=HTMLResponse)
async def login_stub(request: Request) -> HTMLResponse:
    """Заглушка авторизации: выбор роли без реальных паролей."""
    return templates.TemplateResponse(
        request,
        "auth_login.html",
        {"title": "Вход — МФЦ"},
    )
@router.get("/logout", response_class=HTMLResponse)
async def logout_stub(request: Request) -> HTMLResponse:
    return templates.TemplateResponse(
        request,
```

```
        "auth_login.html",
        {"title": "Выход"},
    )
```

2. Модели SqlAlchemy (models.py)

```
class ServiceCategory(Base):
    __tablename__ = "service_categories"

    id: Mapped[str] = mapped_column(String(36), primary_key=True)
    name: Mapped[str] = mapped_column(String(255), nullable=False)
    sort_order: Mapped[int] = mapped_column(Integer, default=0, nullable=False)
class ServiceType(Base):
    __tablename__ = "service_types"

    id: Mapped[str] = mapped_column(String(36), primary_key=True)
    name: Mapped[str] = mapped_column(String(255), nullable=False)
    avg_minutes: Mapped[int] = mapped_column(Integer, default=15, nullable=False)
    priority: Mapped[int] = mapped_column(Integer, default=0, nullable=False)
    requires_appointment: Mapped[bool] = mapped_column(Boolean, default=False, nullable=False)
    fee_rubles: Mapped[int] = mapped_column(Integer, default=0, nullable=False)
    category_id: Mapped[str | None] = mapped_column(
        String(36), ForeignKey("service_categories.id"), nullable=True
    )
    audience: Mapped[str] = mapped_column(String(32), default="residents", nullable=False)
    popularity_rank: Mapped[int] = mapped_column(Integer, default=0, nullable=False)
    terminal_icon: Mapped[str | None] = mapped_column(String(128), nullable=True)

    tickets: Mapped[list["Ticket"]] = relationship(back_populates="service")
class Operator(Base):
    __tablename__ = "operators"

    id: Mapped[str] = mapped_column(String(36), primary_key=True)
    full_name: Mapped[str] = mapped_column(String(255), nullable=False)
    role: Mapped[str] = mapped_column(String(64), default="Оператор", nullable=False)
```

```
    windows: Mapped[list["QueueWindow"]] = relationship(back_populates="operator")
class Client(Base):
    __tablename__ = "clients"
    id: Mapped[str] = mapped_column(String(36), primary_key=True)
    category: Mapped[str] = mapped_column(String(32), default="Обычный", nullable=False)
    last_name: Mapped[str] = mapped_column(String(128), default="", nullable=False)
    first_name: Mapped[str] = mapped_column(String(128), default="", nullable=False)
    middle_name: Mapped[str] = mapped_column(String(128), default="", nullable=False)
    phone: Mapped[str] = mapped_column(String(32), default="", nullable=False)
    email: Mapped[str] = mapped_column(String(255), default="", nullable=False)

    tickets: Mapped[list["Ticket"]] = relationship(back_populates="client")

class QueueWindow(Base):
    __tablename__ = "queue_windows"
    number: Mapped[int] = mapped_column(Integer, primary_key=True)
    status: Mapped[str] = mapped_column(String(32), default="Свободно", nullable=False)
    operator_id: Mapped[str | None] = mapped_column(String(36), ForeignKey("operators.id"), nullable=True)
    service_ids: Mapped[list[str]] = mapped_column(JSON, default=list)
    operator: Mapped[Operator | None] = relationship(back_populates="windows")
class Ticket(Base):
    __tablename__ = "tickets"
    id: Mapped[str] = mapped_column(String(36), primary_key=True)
    number: Mapped[str] = mapped_column(String(32), nullable=False, index=True)
    service_id: Mapped[str] = mapped_column(String(36), ForeignKey("service_types.id"), nullable=False)
    client_id: Mapped[str | None] = mapped_column(String(36), ForeignKey("clients.id"), nullable=True)
    window_number: Mapped[int | None] = mapped_column(Integer, nullable=True)
    created_at: Mapped[datetime] = mapped_column(DateTime, nullable=False)
    slot_at: Mapped[datetime | None] = mapped_column(DateTime, nullable=True)
    status: Mapped[str] = mapped_column(String(64), nullable=False)
    priority: Mapped[int] = mapped_column(Integer, default=0, nullable=False)
    payment_confirmed: Mapped[bool] = mapped_column(Boolean, default=False, nullable=False)
    notes: Mapped[str | None] = mapped_column(Text, nullable=True)
    service: Mapped[ServiceType] = relationship(back_populates="tickets")
```

```
    client: Mapped[Client | None] = relationship(back_populates="tickets")
```

3. Работа с БД (database.py)

```
DATABASE_URL = "sqlite:///./mfc_queue.db"
engine = create_engine(
    DATABASE_URL,
    connect_args={"check_same_thread": False},
)
SessionLocal = sessionmaker(bind=engine, autoflush=False, autocommit=False, expire_on_commit=False)
class Base(DeclarativeBase):
    pass
def get_db() -> Generator[Session, None, None]:
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

4. Основная бизнес-логика (services.py)

```
from .domain import (
    Client,
    ClientCategory,
    Ticket,
    TicketStatus,
    WindowStatus,
    validate_email,
    validate_phone,
)
from .store import MFCStore
class QueueError(Exception):
    pass
@dataclass
class TicketService:
    store: MFCStore
    def create_walk_in_ticket(
        self,
```

```
    service_id: str,
    *,
    paid_fee: bool = False,
    category: ClientCategory = ClientCategory.REGULAR,
) -> Ticket:
    svc = self.store.services.get(service_id)
    if not svc:
        raise QueueError("Услуга не найдена")
    if svc.requires_appointment:
        raise QueueError("Эта услуга только по предварительной записи")
    if self.store.issuance_closed:
        raise QueueError("Выдача талонов завершена, приходите позже")
    waiting = len(self.store.waiting_tickets_for_service(service_id))
    if waiting >= self.store.max_queue_per_service:
        raise QueueError("Очередь переполнена")
    if svc.fee_rubles > 0 and not paid_fee:
        raise QueueError("Требуется оплата госпошлины")
    num = self.store.next_ticket_number()
    priority = 1 if category == ClientCategory.PRIVILEGED else 0
    ticket = Ticket(
        number=num,
        service_id=service_id,
        created_at=datetime.now(),
        status=TicketStatus.WAITING,
        priority=priority,
        payment_confirmed=svc.fee_rubles == 0 or paid_fee,
    )
    self.store.tickets[ticket.id] = ticket
    return ticket
def confirm_payment_stub(self, service_id: str, amount_rubles: int) -> bool:
    svc = self.store.services.get(service_id)
    if not svc:
        return False
    if svc.fee_rubles <= 0:
```

```
            return True
        return amount_rubles >= svc.fee_rubles
    def prebook(
        self,
        service_id: str,
        slot_at: datetime,
        last_name: str,
        first_name: str,
        phone: str,
        email: str,
    ) -> Ticket:
        svc = self.store.services.get(service_id)
        if not svc:
            raise QueueError("Услуга не найдена")
        if not validate_phone(phone):
            raise QueueError("Неверный формат телефона")
        if not validate_email(email):
            raise QueueError("Неверный формат email")
        slots = self.store.available_slots(service_id)
        if slot_at not in slots:
            raise QueueError("Это время уже занято или недоступно")

        num = self.store.next_ticket_number()
        client = Client.new(
            last_name=last_name,
            first_name=first_name,
            phone=phone,
            email=email,
        )
        ticket = Ticket(
            number=num,
            service_id=service_id,
            created_at=datetime.now(),
            status=TicketStatus.PREBOOKED,
```

```
            slot_at=slot_at,
            client=client,
            priority=10,
            payment_confirmed=True,
        )
        self.store.tickets[ticket.id] = ticket
        return ticket
@dataclass
class QueueManager:
    store: MFCStore
    ticket_service: TicketService

    def get_ticket_status(self, number: str) -> Optional[Tuple[Ticket, Optional[int]]]:
        for t in self.store.tickets.values():
            if t.number == number:
                return t, t.window_number
        return None

    def board_snapshot(self) -> list[dict]:
        out: list[dict] = []
        for t in self.store.tickets.values():
            if t.status in (TicketStatus.CALLED, TicketStatus.SERVING) and t.window_number:
                out.append({"number": t.number, "window": t.window_number, "status": t.status.value})
        out.sort(key=lambda x: x["number"])
        return out

    def waiting_queue_snapshot(self) -> list[dict]:
        """Очередь на табло: живая очередь + все предварительные записи (с временем приёма)."""
        now = datetime.now()
        rows: list[Ticket] = []
        for t in self.store.tickets.values():
            if t.status == TicketStatus.WAITING:
                rows.append(t)
            elif t.status == TicketStatus.PREBOOKED and t.slot_at:
```

```
            rows.append(t)

        def sort_key(t: Ticket) -> tuple:
            if t.status == TicketStatus.PREBOOKED and t.slot_at:
                if t.slot_at <= now:
                    return (0, -t.priority, t.slot_at)
                return (2, -t.priority, t.slot_at)
            return (1, -t.priority, t.created_at)

        rows.sort(key=sort_key)
        out: list[dict] = []
        for pos, t in enumerate(rows, start=1):
            svc = self.store.services.get(t.service_id)
            slot_iso = t.slot_at.isoformat(timespec="minutes") if t.slot_at else None
            slot_hm = t.slot_at.strftime("%d.%m.%Y %H:%M") if t.slot_at else ""
            due = bool(t.slot_at and t.slot_at <= now) if t.status == TicketStatus.PREBOOKED else True
            out.append(
                {
                    "position": pos,
                    "number": t.number,
                    "service_name": svc.name if svc else "",
                    "status": t.status.value,
                    "slot_hm": slot_hm,
                    "slot_iso": slot_iso,
                    "is_prebook": t.status == TicketStatus.PREBOOKED,
                    "is_due": due,
                }
            )
        return out
    def call_next(self, window_number: int) -> Optional[Ticket]:
        window = self.store.windows.get(window_number)
        if not window:
            raise QueueError("Окно не найдено")
        if window.current_ticket_id or window.status == WindowStatus.BUSY:
```

```
    raise QueueError("Окно уже занято активным талоном")

candidates: list[Ticket] = []
seen: set[str] = set()
for sid in window.service_ids:
    for t in self.store.waiting_tickets_for_service(sid):
        if t.id not in seen:
            seen.add(t.id)
            candidates.append(t)
for t in self.store.tickets.values():
    if (
        t.id not in seen
        and t.status == TicketStatus.PREBOOKED
        and t.slot_at
        and t.service_id in window.service_ids
    ):
        candidates.append(t)
        seen.add(t.id)

now = datetime.now()

def call_order(t: Ticket):
    if t.status == TicketStatus.PREBOOKED and t.slot_at:
        if t.slot_at <= now:
            return (0, -t.priority, t.slot_at)
        return (2, -t.priority, t.slot_at)
    return (1, -t.priority, t.created_at)

candidates.sort(key=call_order)
if not candidates:
    return None

ticket = candidates[0]
ticket.status = TicketStatus.CALLED
```

```
    ticket.window_number = window_number
    window.status = WindowStatus.BUSY
    window.current_ticket_id = ticket.id

    self.store.board.last_calls.insert(0, (ticket.number, window_number))
    self.store.board.last_calls = self.store.board.last_calls[: self.store.board.max_visible]
    return ticket

def start_serving(self, window_number: int) -> Ticket:
    window = self.store.windows.get(window_number)
    if not window or not window.current_ticket_id:
        raise QueueError("Нет активного талона")
    t = self.store.tickets[window.current_ticket_id]
    if t.status not in (TicketStatus.CALLED, TicketStatus.SERVING):
        raise QueueError("Неверный статус талона")
    t.status = TicketStatus.SERVING
    return t

def complete_service(self, window_number: int, success: bool) -> None:
    window = self.store.windows.get(window_number)
    if not window or not window.current_ticket_id:
        raise QueueError("Нет активного талона")
    t = self.store.tickets[window.current_ticket_id]
    if t.status not in (TicketStatus.CALLED, TicketStatus.SERVING):
        raise QueueError("Неверный статус талона для завершения")
    t.status = TicketStatus.COMPLETED if success else TicketStatus.REJECTED
    window.status = WindowStatus.FREE
    window.current_ticket_id = None

def no_show(self, window_number: int) -> None:
    window = self.store.windows.get(window_number)
    if not window or not window.current_ticket_id:
        raise QueueError("Нет активного талона")
    t = self.store.tickets[window.current_ticket_id]
```

```
        if t.status not in (TicketStatus.CALLED, TicketStatus.SERVING):
            raise QueueError("Неверный статус для неявки")
        t.status = TicketStatus.NO_SHOW
        window.status = WindowStatus.FREE
        window.current_ticket_id = None

    def redirect(self, window_number: int, new_service_id: str) -> None:
        window = self.store.windows.get(window_number)
        if not window or not window.current_ticket_id:
            raise QueueError("Нет активного талона")
        new_svc = self.store.services.get(new_service_id)
        if not new_svc:
            raise QueueError("Услуга не найдена")

        if not any(new_service_id in ww.service_ids for ww in self.store.windows.values()):
            raise QueueError("Специалисты по данной услуге недоступны")

        t = self.store.tickets[window.current_ticket_id]
        t.service_id = new_service_id
        t.priority += 100
        t.status = TicketStatus.WAITING
        t.window_number = None
        window.status = WindowStatus.FREE
        window.current_ticket_id = None
```

5. Пример роутера Оператора

```
router = APIRouter(tags=["operator"])
def _services_sorted():
    return sorted(store.services.values(), key=lambda s: s.name)
@router.get("/operator", response_class=HTMLResponse)
async def operator_dashboard(request: Request) -> HTMLResponse:
    return templates.TemplateResponse(
        request,
        "operator_dashboard.html",
        {
```

```
            "title": "Оператор — МФЦ",
            "windows": sorted(store.windows.keys()),
        },
    )
@router.get("/operator/panel", response_class=HTMLResponse)
async def operator_panel(
    request: Request,
    window: int = 1,
    msg: str | None = None,
    err: str | None = None,
    focus: str | None = None,
) -> HTMLResponse:
    w = store.windows.get(window)
    current = None
    if w and w.current_ticket_id:
        current = store.tickets.get(w.current_ticket_id)
    return templates.TemplateResponse(
        request,
        "operator_panel.html",
        {
            "windows": sorted(store.windows.keys()),
            "window": window,
            "win": w,
            "current": current,
            "services": _services_sorted(),
            "msg": msg,
            "err": err,
            "focus": focus,
        },
    )
@router.post("/operator/call")
async def op_call(window: int = Form(...)):
    try:
        t = queue_manager.call_next(window)
```

```
    except QueueError as e:
        return RedirectResponse(f"/operator/panel?window={window}&err={quote(str(e))}", status_code=303)
    if not t:
        return RedirectResponse(f"/operator/panel?window={window}&msg=Очередь+пуста", status_code=303)
    return RedirectResponse(f"/operator/panel?window={window}&msg=Вызван+{t.number}", status_code=303)


@router.post("/operator/start")
async def op_start(window: int = Form(...)):
    try:
        queue_manager.start_serving(window)
    except QueueError as e:
        return RedirectResponse(f"/operator/panel?window={window}&err={quote(str(e))}", status_code=303)
    return RedirectResponse(f"/operator/panel?window={window}&msg=Обслуживание+начато",
status_code=303)



@router.post("/operator/complete")
async def op_complete(window: int = Form(...), success: str = Form("yes")):
    ok = success == "yes"
    try:
        queue_manager.complete_service(window, ok)
    except QueueError as e:
        return RedirectResponse(f"/operator/panel?window={window}&err={quote(str(e))}", status_code=303)
    return RedirectResponse(f"/operator/panel?window={window}&msg=Завершено", status_code=303)


@router.post("/operator/noshow")
async def op_noshow(window: int = Form(...)):
    try:
        queue_manager.no_show(window)
    except QueueError as e:
        return RedirectResponse(f"/operator/panel?window={window}&err={quote(str(e))}", status_code=303)
    return RedirectResponse(f"/operator/panel?window={window}&msg=Неявка", status_code=303)


@router.post("/operator/redirect")
```

```
async def op_redirect(window: int = Form(...), new_service_id: str = Form(...)):
    try:
        queue_manager.redirect(window, new_service_id)
    except QueueError as e:
        return RedirectResponse(f"/operator/panel?window={window}&err={quote(str(e))}", status_code=303)
    return RedirectResponse(f"/operator/panel?window={window}&msg=Перенаправлен", status_code=303)
```

4. Этап тестирования

4.1 План тестирования

Цель тестирования:

- проверка корректности работы системы;
- проверка обработки ошибок;
- проверка пользовательских сценариев.

4.2 Тестовые сценарии

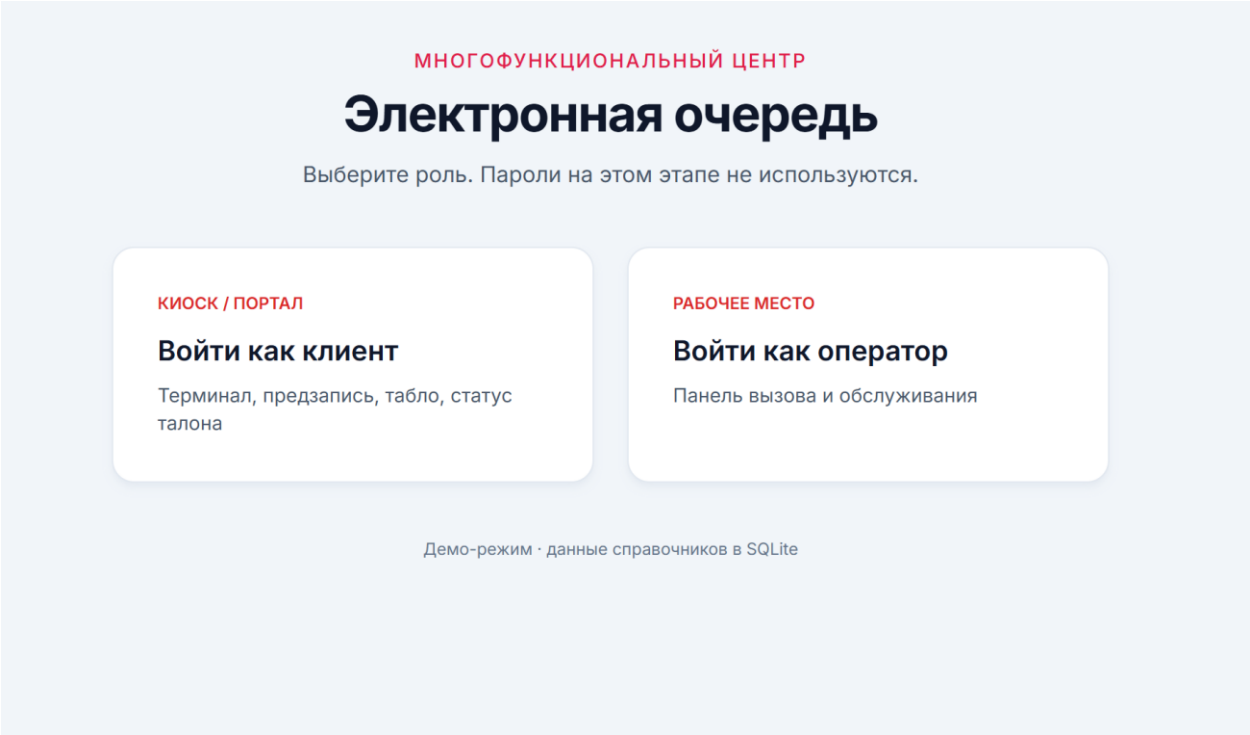
№	Сценарий	Ожидаемый результат
TC-01	Получение талона	Создан новый талон
TC-02	Вызов клиента	Статус меняется на «Вызван»
TC-03	Завершение обслуживания	Статус меняется на «Завершён»
TC-04	Неявка клиента	Статус «Неявка»

4.3 Протокол тестирования

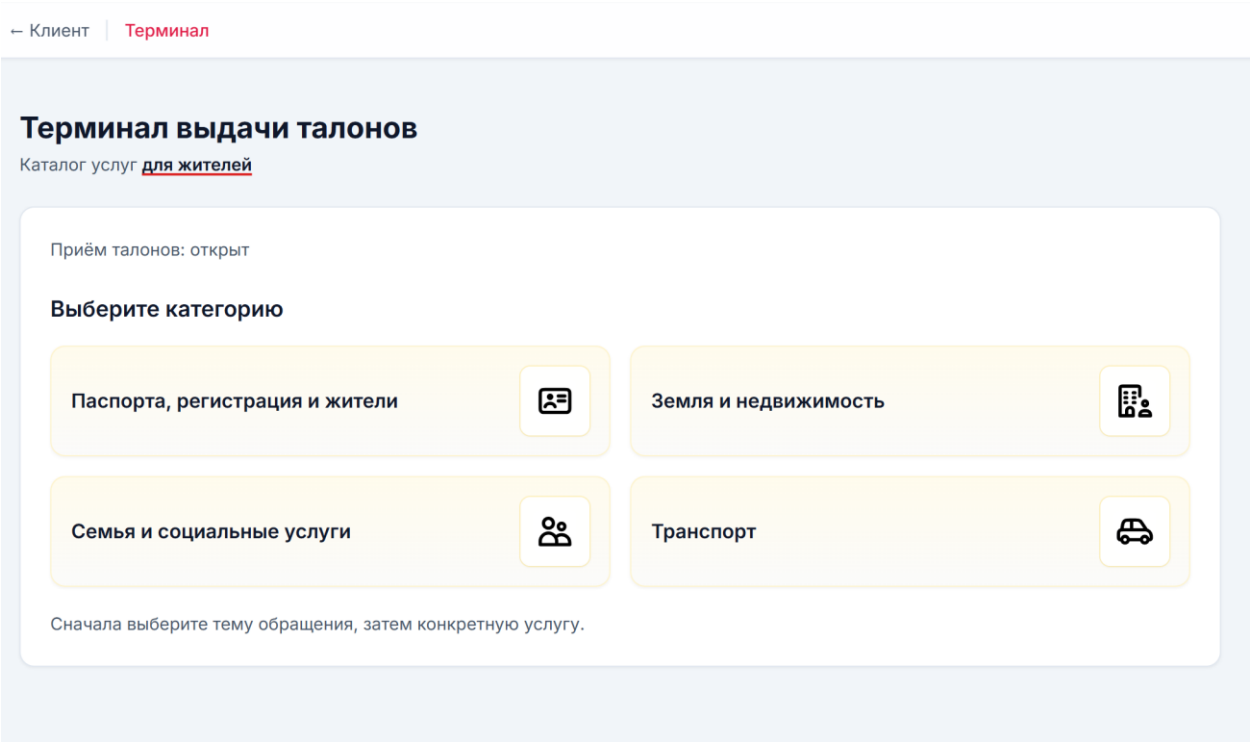
Тест	Результат
TC-01	Успешно
TC-02	Успешно
TC-03	Успешно
TC-04	Успешно

4.4 Скриншоты работы программы

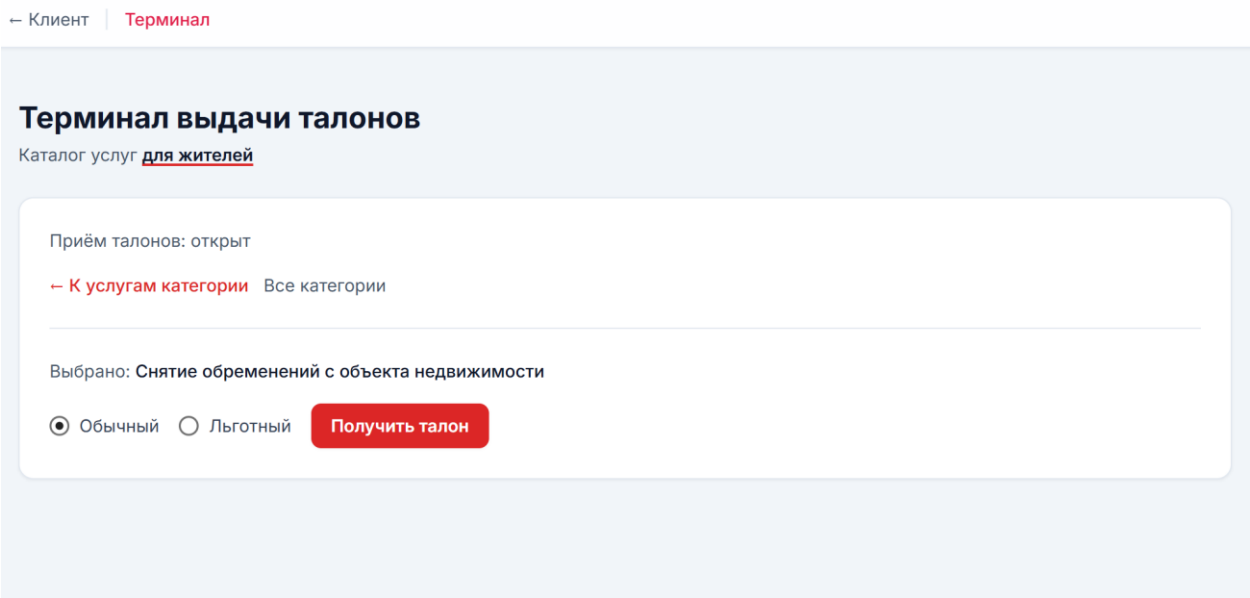
В раздел добавляются:



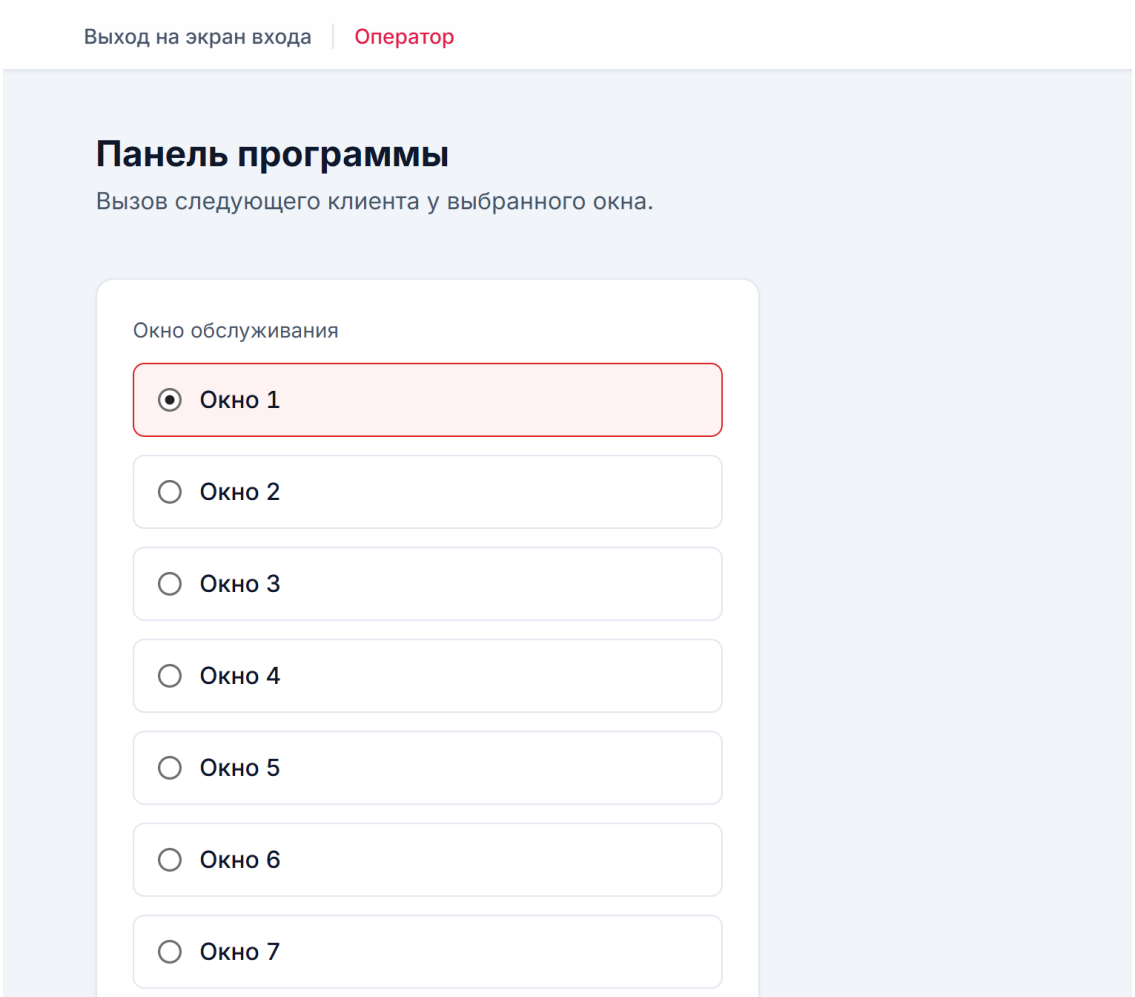
Главное меню

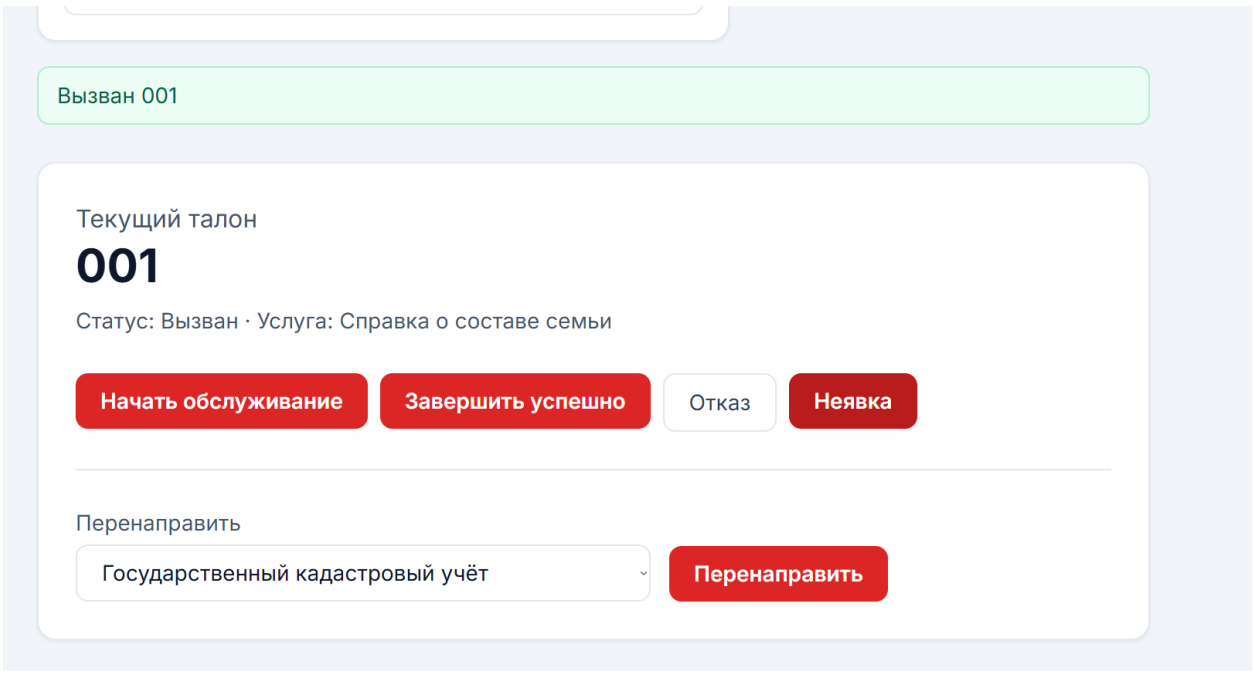


Выбор услуги

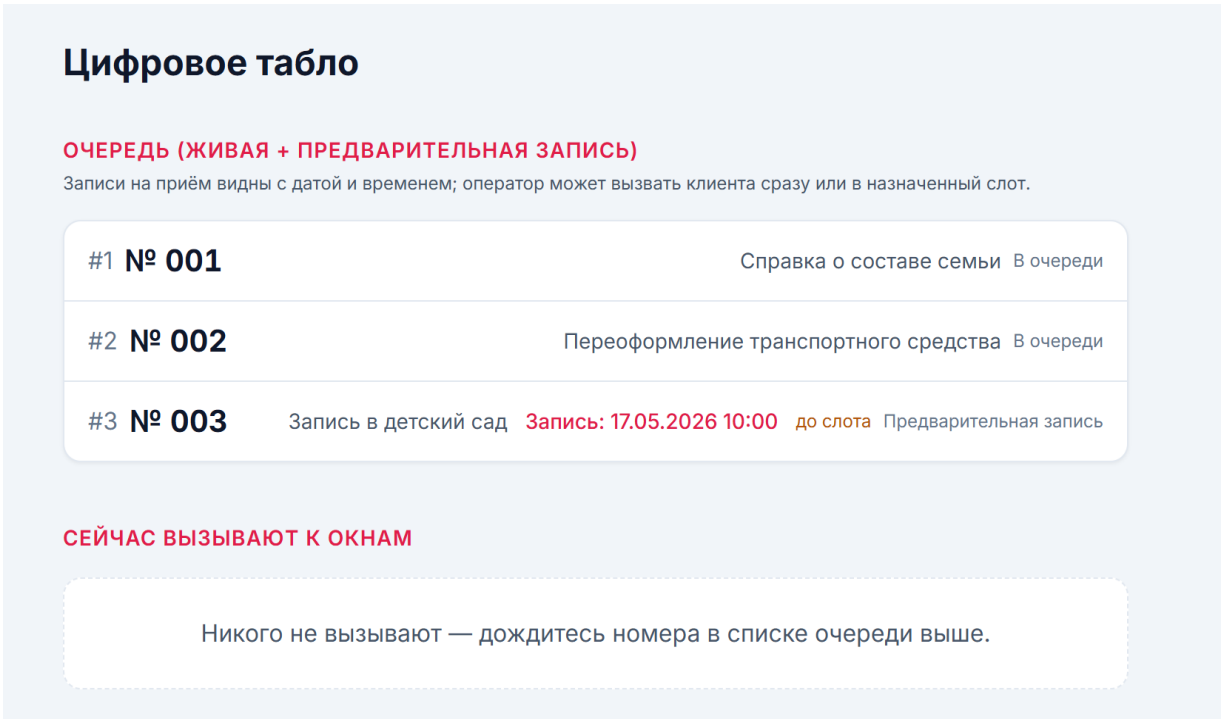


Экран получения талона

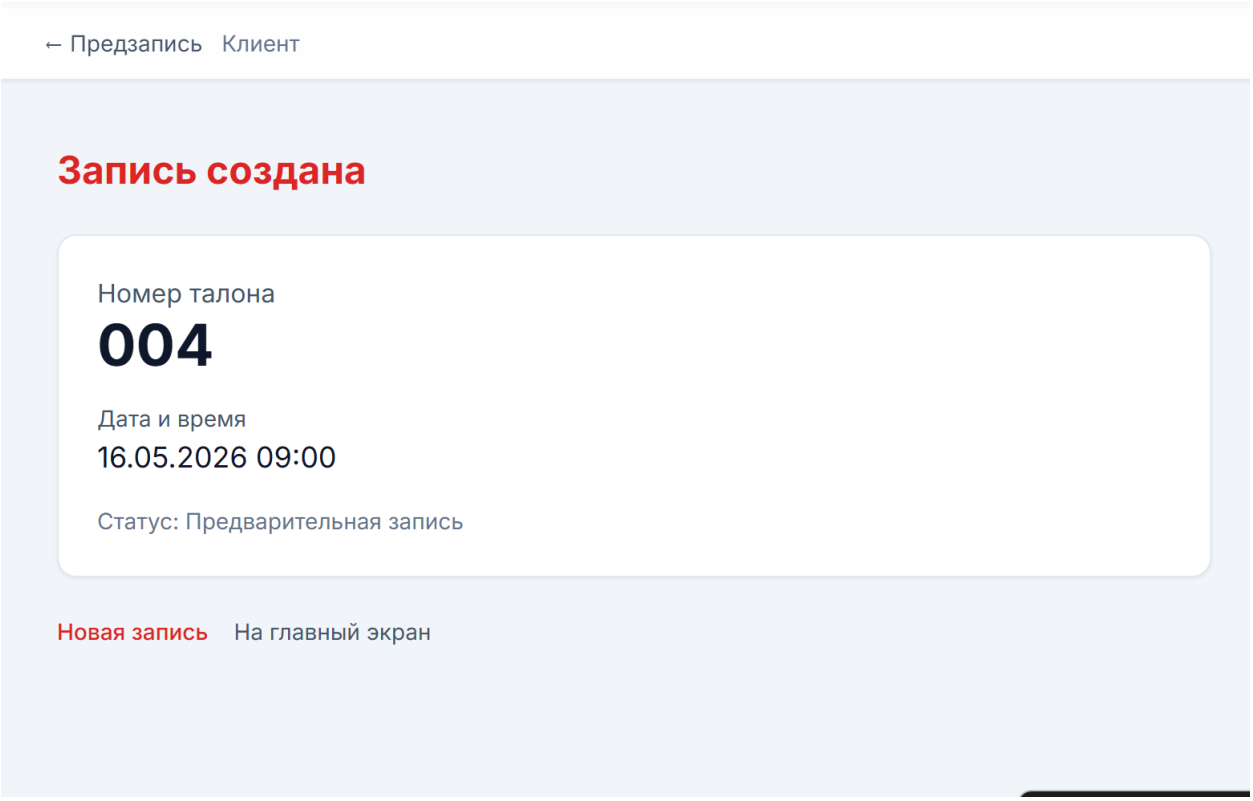




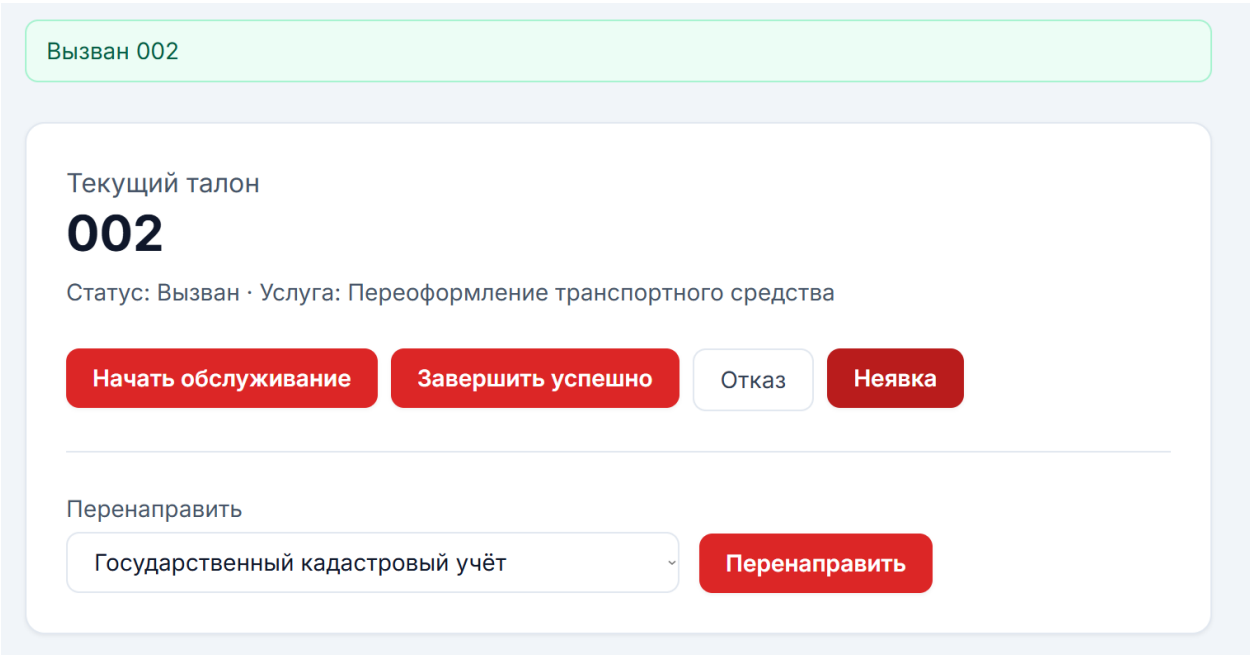
Окно оператора



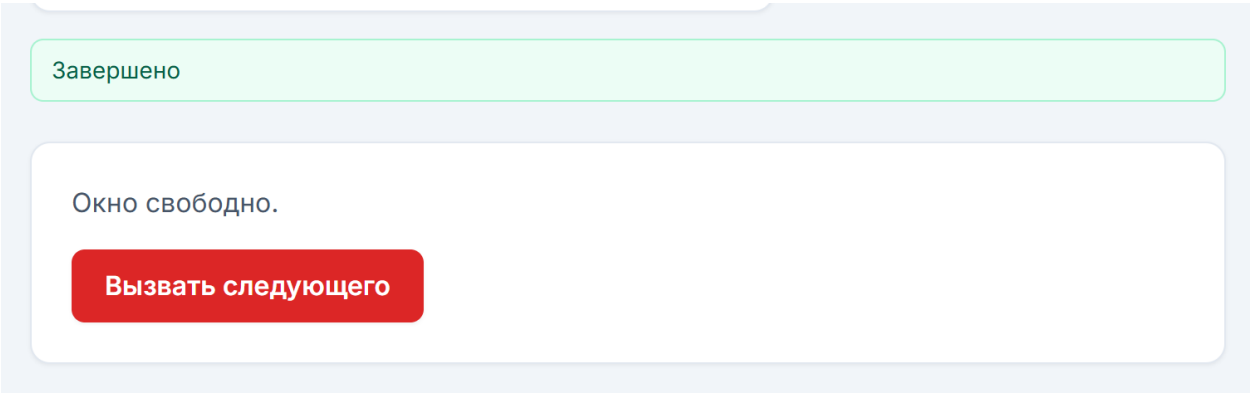
Цифровое табло



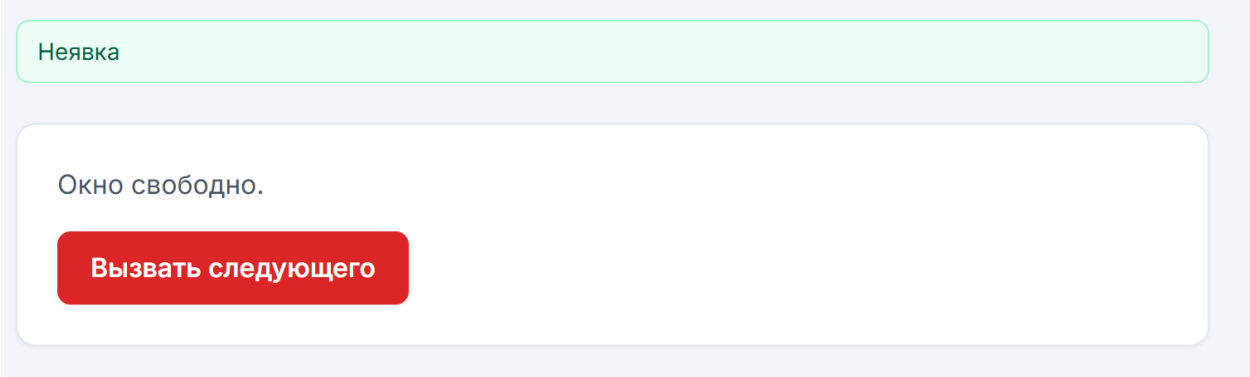
ТС-01 Получение талона



ТС-02 Вызов клиента



ТС-03 Завершение обслуживания



ТС-04 Неявка клиента

5. ЗАКЛЮЧЕНИЕ

В ходе выполнения работы была разработана система электронной очереди для МФЦ. Выполнены анализ требований, проектирование UML-моделей, реализация программного обеспечения и тестирование системы.

Разработанная система позволяет автоматизировать процесс обслуживания клиентов, сократить время ожидания и повысить эффективность работы операторов.

GtHub: https://github.com/vwpip/MFC_electronic_queue